Social Media Content Editor Agent

Engineering Handoff v4 — all gaps resolved

Repo	GenAI-Showcase/apps/video-intelligence https://github.com/mongodb-developer/GenAI-Showcase/tree/main/apps/video-intelligence
Purpose	Keynote demo for MongoDB .local London
Extends	Existing FastAPI + React video search app
Owner	Mikiko Bazeley & Frank Liu
Version	v4 — April 16, 2026

1. The Interaction Flow

The editor opens a living project workspace where the agent has already been working. The editor refines, doesn't start from scratch. The demo uses two pre-seeded projects to show cross-project memory learning.

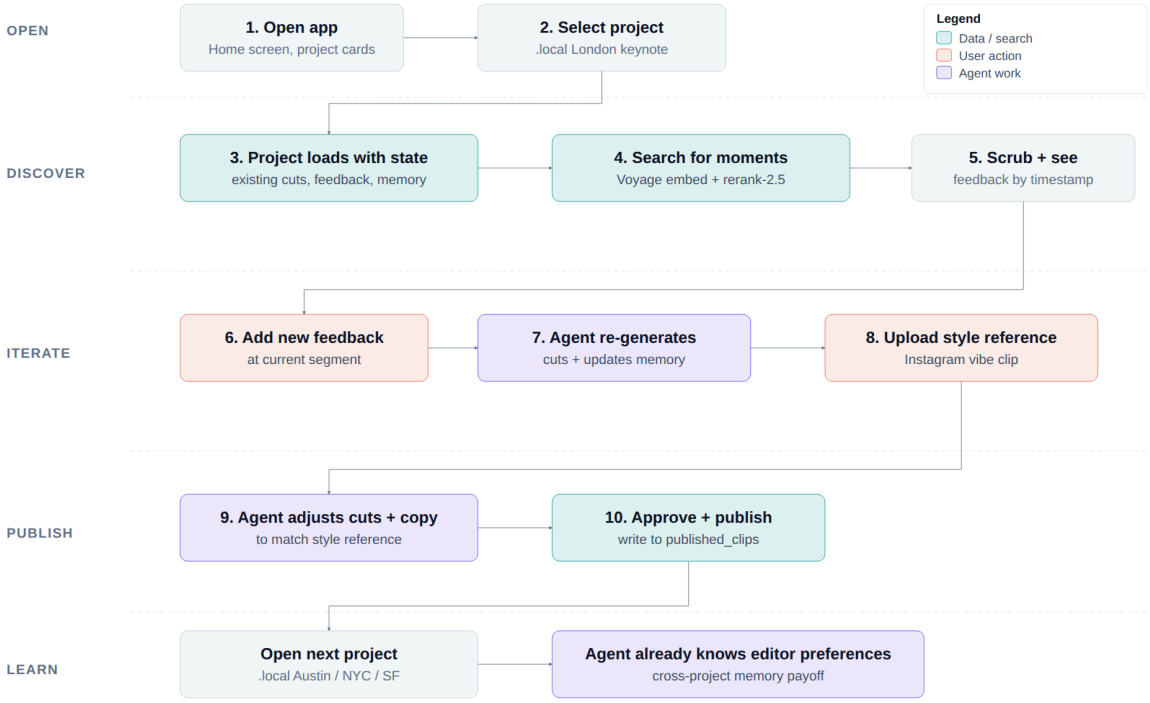

Figure 1: Demo interaction flow — 10 steps from project open to cross-project memory payoff

Step by step

1. Open app, select project. Home screen shows project cards. Four pre-loaded: Austin (published), NYC (published), SF (published), London (in progress). Start with London as the main workspace.

2. Project loads with existing state. Video player, timeline with feedback markers, agent's current recommended cuts (persisted from prior run), memory sidebar.

3. Search for moments. Editor types a query. The Keynote Agent routes to hybrid search (Voyage multimodal embeddings + full-text on transcript + Voyage rerank-2.5). Results appear as a thumbnail strip. This is where the Voyage embedding model shines.

4. Scrub video, see feedback by timestamp. As the video plays or scrubs, the feedback panel updates to show comments covering the current timestamp. Conflict markers are orange.

5. Provide new feedback. Editor pauses, types feedback. The timestamp range defaults to the current segment boundaries. Submitting triggers the Social Agent to re-generate.

6. Social Agent re-generates cuts + updates memory. The Social Agent re-runs with updated feedback. If there's a conflict, the agent pauses via interrupt() and asks the editor to resolve. The agent also analyzes feedback patterns and updates style_memory.

7. Upload style reference. Editor drops an Instagram clip. System extracts frames, Claude analyzes style (pacing, hook structure, caption tone). Style matching = timing + copy tone + format recs, not visual transformation.

8. Review and approve. Editor reviews cuts with editable captions, suggested hashtags, and asset recommendations. Approves some, rejects others.

9. Publish. Approved clips written to published_clips. Content memory and style memory updated. Confirmation shown.

10. Switch to another project. Editor opens a different project. The agent's recommended cuts already reflect preferences learned from previous projects — this is the cross-project memory payoff.

2. System Architecture

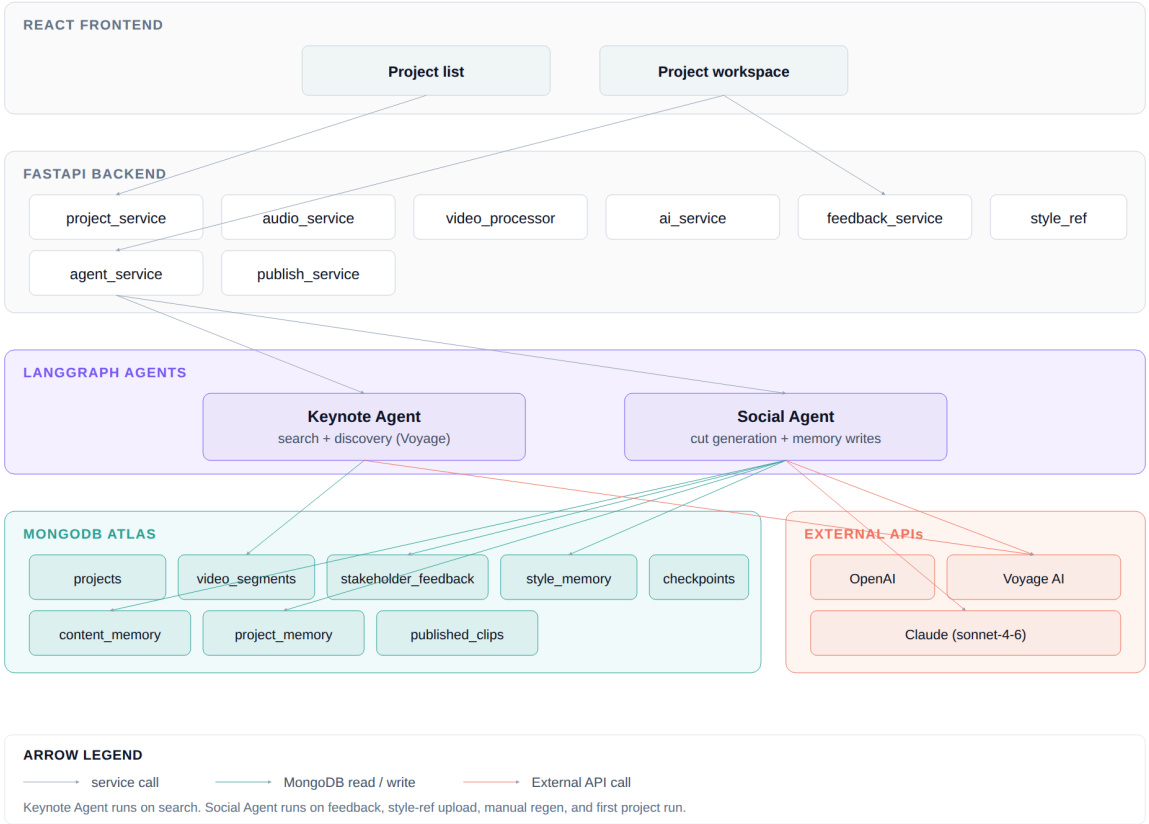

Figure 2: System architecture — React frontend, LangGraph agents, FastAPI backend, MongoDB Atlas, external APIs

Agent	Role	When it runs
Keynote Agent: Search	Takes natural language queries, embeds via Voyage, routes to hybrid search, reranks with rerank-2.5. This is the Voyage showcase.	When editor types a search query
Social Agent: Cut Generation	Takes segments + feedback + memory + style reference, produces cut list with per-platform captions, hashtags, and asset recommendations. Detects conflicts, triggers HITL. Updates memory.	On feedback submission, style reference upload, manual regenerate, and initial project creation

3. Data Flow Diagrams

3a. Project load

Figure 3: Project load — single compound read returns full workspace state

3b. Search for moments (Keynote Agent)

Figure 4: Search flow — Voyage embed → MongoDB hybrid search → Voyage rerank-2.5 → results

3c. New feedback triggers re-generation

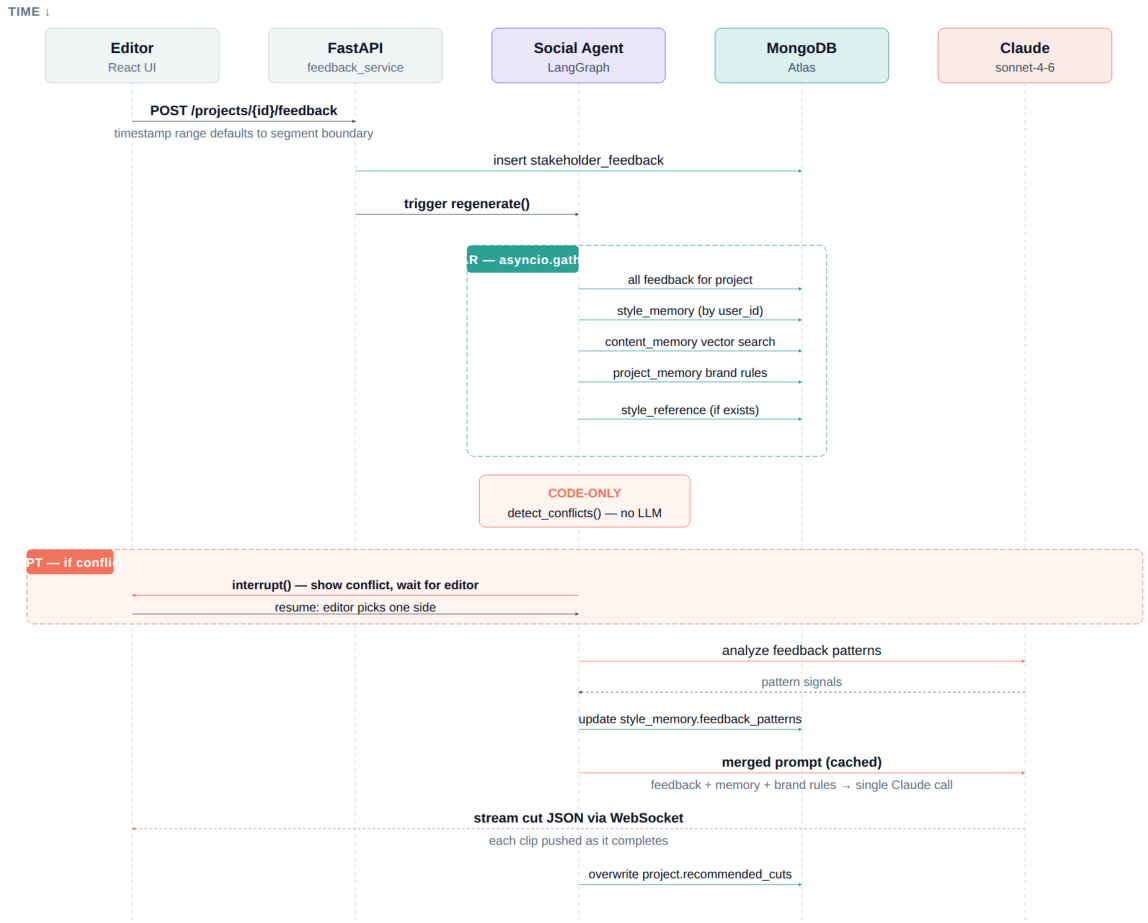

Figure 5: Feedback → parallel memory reads → conflict detection → merged Claude prompt → streamed cut list

3d. Style reference upload

Figure 6: Style reference — extract frames → Voyage embed → Claude analyze → trigger re-generation

3e. Publish + cross-project memory

Figure 7: Publish writes to published_clips, content_memory, and style_memory. Same style_memory used when opening next project.

3f. Ingest pipeline (pre-baked)

Figure 8: Ingest — ffmpeg → GPT-4 Vision + gpt-4o-transcribe-diarize → merge → Claude summarize → Voyage embed → Atlas

Video file serving: The existing main.py deletes the uploaded video after processing (video_path.unlink()). Remove this line. Videos must persist in uploads/ and be served via FastAPI static files.

4. Collection Schemas

projects (new — central workspace object)

Bundles video + feedback + cuts + style + campaign into one workspace. recommended_cuts is overwritten (not appended) on each agent run. Doc stays well under MongoDB's 16MB limit.

```
{
  "project_id": "proj_london_keynote",
  "title": "MongoDB .local London 2026 Keynote",
  "video_id": "uuid", "video_path": "uploads/uuid.mp4",
  "campaign_id": "local_london_2026",
  "status": "in_progress",  // draft | in_progress | published
  "duration": 1440.0,
  "segment_boundaries": [
    { "start": 0.0, "end": 34.2, "speaker": "SPEAKER_00" }, ...
  ],
  "recommended_cuts": [
    {
      "clip_id": "cut_001",
      "start_time": 495.0, "end_time": 525.0,
      "thumbnail_path": "/frames/uuid/frame_0248.jpg",
      "transcript_excerpt": "And that's why we built...",
      "speaker": "SPEAKER_01",
      "suggested_caption": {
        "linkedin": "What does it take to build search...",
        "reels": "This is why vector search changes everything"
      },
      "suggested_hashtags": {
        "linkedin": ["#MongoDB", "#VectorSearch", "#AI"],
        "reels": ["#mongodb", "#vectorsearch", "#ai"]
      },
      "asset_recommendation": "Use left camera (CJ pref). Wide shot.",
      "memory_notes": "No dupe. Matches LinkedIn pref 60s.",
      "status": "pending"
    }
  ],
  "style_reference": null,
  "created_at": "2026-04-14T10:00:00Z",
  "last_agent_run": "2026-04-15T14:00:00Z"
}
```

stakeholder_feedback (new)

author is a plain string, not tied to any auth system.

```
{
  "feedback_id": "fb_001", "project_id": "proj_london",
  "video_id": "uuid",
  "timestamp_range": { "start": 142.3, "end": 158.7 },
  "author": "CJ",
  "comment_text": "Make sure you get my good side",
  "created_at": "2026-04-10T14:00:00Z"
}
```

style_memory (new — shared across all projects)

Keyed by user_id, not project_id. This is how cross-project memory works.

```
{
  "user_id": "editor_mikiko",
  "preferred_clip_lengths": { "reels": 15, "linkedin": 60, "twitter": 30 },
  "pacing": "fast_cuts", "hook_style": "question_first",
  "feedback_patterns": [
    "consistently flags pacing issues for Reels",
    "prefers question-first hooks",
    "favors audience reaction shots"
  ],
  "accepted_cuts": ["austin_clip_001", ... , "sf_clip_003"],
  "rejected_cuts": ["austin_draft_003", "nyc_draft_003", "sf_draft_004"],
  "updated_at": "2026-03-17T12:00:00Z"
}
```

project_memory (new — with speaker profiles)

```
{
  "campaign_id": "local_london_2026",
  "event_name": "MongoDB .local London 2026",
  "brand_voice_rules": "Professional but approachable. No jargon.",
  "previous_campaigns": ["local_austin_2025", "local_nyc_2025", "local_sf_2026"],
  "target_platforms": ["linkedin", "twitter", "reels"],
  "speaker_profiles": {
    "CJ": {
      "preferred_angle": "left camera",
      "voice_notes": "Conversational, uses developer analogies",
      "quirks": ["always says 'and that's the thing' before key points"]
```

```
    },
    "Oz": {
      "voice_notes": "Technical, precise, enterprise framing",
      "quirks": []
    }
  }
}
```

Entity relationships

Figure 9: ER diagram — projects at center, style_memory shared across projects, content_memory for dupe checks

5. Memory Types & Cross-Project Learning

Memory type	Collection	Scope	When it updates
Working	MongoDBSaver	Per session	Every agent step (auto)
Shared	stakeholder_feedback	Per project	Any reviewer submits
Episodic + Procedural	style_memory	**Per editor, across all projects**	On feedback, accept/reject, publish
Semantic	project_memory	Per campaign	Pre-seeded
Associative	content_memory	**Global, across all projects**	On publish

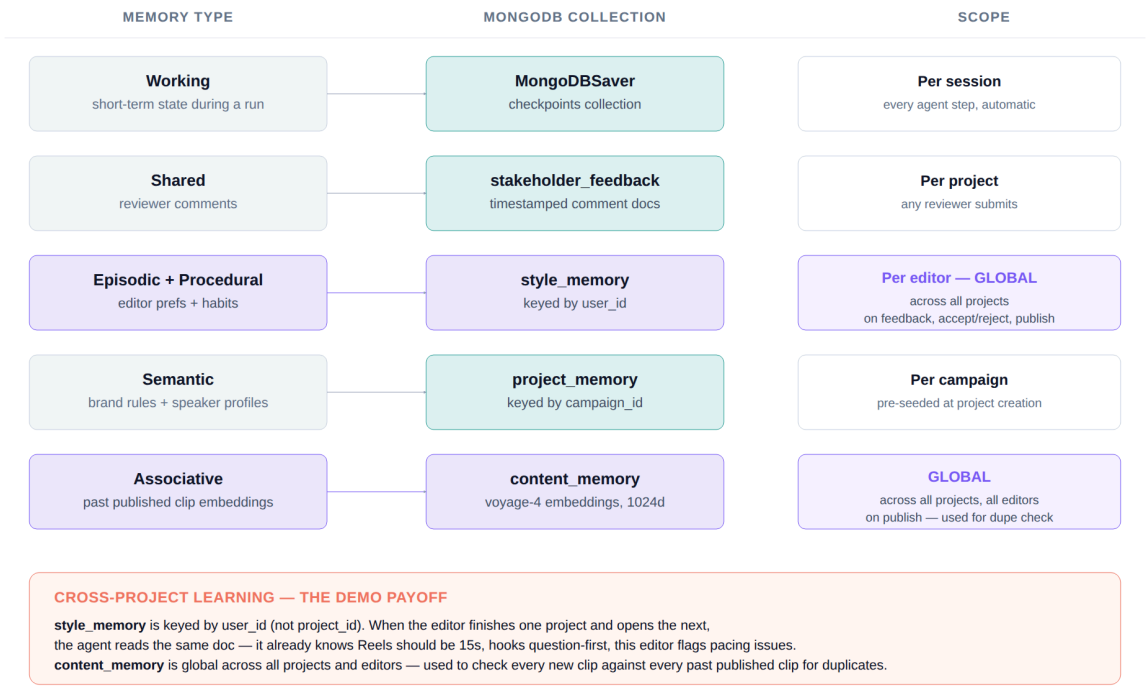

Figure 10: Memory types → MongoDB collections. style_memory and content_memory are global (highlighted) — this is why the agent gets better over time.

The cross-project demo moment: style_memory is keyed by user_id, not project_id. When the editor finishes project A and opens project B, the agent reads the same style_memory doc. content_memory is also global — the agent checks all past published clips across all projects for duplicates.

6. Agent Prompts & Intelligence

Social Agent: System prompt (skeleton)

This is the cached system prompt block. It stays the same across calls within a session, so Anthropic's prompt caching avoids re-processing it after the first call.

```
You are the Social Agent — a social media content editor agent. You analyze keynote video segments
and produce clip recommendations with per-platform captions.

CONTEXT PROVIDED TO YOU:
- Video segments with transcripts and speaker labels
- Stakeholder feedback (timestamped comments from multiple reviewers)
- Editor preferences from style_memory (clip lengths, pacing, hook style)
- Brand voice rules from project_memory
- Speaker profiles from project_memory (voice notes, preferred angles, quirks)
- Content memory (previously published clips — avoid duplicates)
```

```
- Style reference analysis (if uploaded — match pacing and tone)

CONFLICT RESOLUTION:
When two reviewers give contradictory feedback on the same segment,
DO NOT resolve it yourself. Present both positions and ask the editor
to decide. Format: "Conflict at [timestamp]: [Author A] says [X],
[Author B] says [Y]. Which direction do you prefer?"

CLIP GENERATION RULES:
1. Respect editor's preferred clip lengths per platform from style_memory
2. Check every candidate clip against content_memory — if cosine similarity
   > 0.85 with any published clip, flag it with a warning
3. Apply brand_voice_rules to all captions
4. If speaker_profiles exist for the speaker in a clip, incorporate their
   preferences into memory_notes (e.g., preferred camera angle) and match
   their voice/tone in the caption copy
5. If a style_reference exists, match its pacing and tone:
   - Pacing: adjust clip length and cut frequency
   - Tone: adjust caption formality, emoji usage, hook structure
   - Format: note aspect ratio and text overlay recommendations
6. For each clip, include memory_notes explaining your reasoning
7. Include suggested_hashtags (3-5 relevant hashtags per platform)
8. Include asset_recommendation (what visual treatment would make it land)

RESPONSE FORMAT:
Respond with a single JSON object containing exactly three keys:

{
  "conflicts": [
    {"timestamp": "14:20-15:00", "authors": ["CJ", "Oz"],
     "positions": ["Cut — misspoke", "Keep — correction is relatable"]}
  ],
  "feedback_patterns": [
    "consistently flags pacing issues for Reels",
    "prefers question-first hooks"
  ],
  "recommended_cuts": [
    {
```

```
      "clip_id": "cut_001",
      "start_time": 495.0, "end_time": 525.0,
      "transcript_excerpt": "...",
      "speaker": "CJ",
      "suggested_caption": {"linkedin": "...", "reels": "..."},
      "suggested_hashtags": {"linkedin": ["#MongoDB", "..."], "reels": ["#mongodb",
"..."]},
      "asset_recommendation": "Use left camera angle (CJ pref). Wide shot with
audience.",
      "memory_notes": "No dupe. Matches LinkedIn 60s pref. CJ prefers left camera.",
      "status": "pending"
    }
  ]
}
```

Why one prompt instead of three: The original spec had separate calls for conflict detection, pattern extraction, and cut list generation. Merging them into one call eliminates two network round-trips to Claude, saving 4-6 seconds. The merged prompt is larger but Claude Sonnet handles structured multi-section output well. See section 11, optimization 1 for the full code pattern.

Feedback pattern detection

Handled inside the merged prompt above (the `feedback_patterns` key). Output is written to `style_memory.feedback_patterns` after each generation run. If you need to run pattern detection separately (e.g., outside of a re-generation), use `claude-haiku-4-5-20251001` for speed:

```
# Standalone pattern detection (only if not using merged prompt):
patterns = claude.messages.create(
    model="claude-haiku-4-5-20251001",  # Haiku for speed
    max_tokens=256,
    messages=[{
        "role": "user",
        "content": f"Analyze these feedback entries from editor \"{user_id}\". "
                   f"Extract 2-3 recurring preference patterns as a JSON array of
short strings.\n\n"
                   f"Feedback:\n{last_10_json}"
    }]
)
```

Style reference analysis prompt

```
Analyze these frames from a social media reference clip.
Describe the style in terms of:
```

```
- pacing: how many cuts per 10 seconds, average shot duration
- hook_structure: how the first 2 seconds grab attention
- caption_tone: formal/casual/playful, emoji usage, sentence length
- format_notes: aspect ratio, text overlay style, any trending patterns

Return as JSON matching: {
  "pacing": "...",
  "hook_structure": "...",
  "caption_tone": "...",
  "format_notes": "..."
}
```

Conflict detection logic (code-only, no LLM call)

For speed, conflict detection uses keyword heuristics only — no Claude call. This runs in microseconds and catches all pre-seeded demo cases. Only fall back to Claude classification in production for genuinely ambiguous cases.

```
CUT_WORDS = {"cut", "remove", "drop", "delete", "skip", "lose"}
KEEP_WORDS = {"keep", "include", "use", "love", "great", "perfect"}


def detect_conflicts(feedback_entries):
    """Group by overlapping timestamp ranges, flag where
    different authors give contradictory direction."""
    conflicts = []
    for i, a in enumerate(feedback_entries):
        for b in feedback_entries[i+1:]:
            if a["author"] != b["author"] and ranges_overlap(a, b):
                words_a = set(a["comment_text"].lower().split())
                words_b = set(b["comment_text"].lower().split())
                a_cuts = bool(words_a & CUT_WORDS)
                b_keeps = bool(words_b & KEEP_WORDS)
                a_keeps = bool(words_a & KEEP_WORDS)
                b_cuts = bool(words_b & CUT_WORDS)
                if (a_cuts and b_keeps) or (a_keeps and b_cuts):
                    conflicts.append({
                        "entries": [a, b],
                        "timestamp": a["timestamp_range"]
                    })
    return conflicts
```

The editor always resolves conflicts manually. The agent does not pick sides.

7. LangGraph Integrations

MongoDBSaver (checkpointer)

```
from langgraph.checkpoint.mongodb import MongoDBSaver
saver = MongoDBSaver.from_conn_string(MONGODB_URI, db_name="video_intelligence")
```

HumanInTheLoopMiddleware

GA in LangChain 1.0. Import from `langchain.agents.middleware` (not `langgraph`). Resume with `Command(resume=...)` from `langgraph.types`.

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.types import Command

agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool, generate_cuts_tool, publish_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "generate_cuts": True,      # pause before generating
                "publish_clips": True,      # pause before publishing
                "search_segments": False,   # auto-approve
            },
            description_prefix="Editor approval needed",
        ),
    ],
    checkpointer=saver,  # MongoDBSaver from above
)

# Run until interrupt
config = {"configurable": {"thread_id": "proj_london_session_1"}}
result = agent.invoke({"messages": [...]}, config=config)

# After editor resolves conflict, resume:
agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config,
```

```
)
```

> **Known issues:** `.ainvoke()` with HITL middleware can throw errors in some environments (GitHub #34974). **Prefer sync .invoke() or .stream() for the keynote.** Also pin `langgraph-prebuilt==1.0.9` explicitly — v1.0.2 had a breaking change.

MongoDBStore (long-term memory)

Use for `style_memory` and `content_memory`. Raw pymongo for `project_memory` (simple reads). Use client-side embeddings (manual Voyage API calls) for the demo — the server-side auto-embed option only works on MongoDB Community 8.2+, not Atlas.

```
# pip install langgraph-checkpoint-mongodb==0.3.1 langgraph-store-mongodb==0.2.0
```

Voyage AI Reranking on search results

After the Keynote Agent retrieves candidate segments via hybrid search, rerank them with Voyage's cross-encoder before returning results. This jointly processes query + document for more accurate relevance scoring than embedding similarity alone. ~3 lines of code in the search pipeline.

```
import voyageai
vo = voyageai.Client()

# After hybrid_search returns candidates:
# NOTE: rerank-2.5 supports instruction-following via query prepend.
# Prepend task-specific instructions to steer relevance scoring.
results = vo.rerank(
    query="Prioritize segments where the speaker directly addresses the audience. "
          + query_text,   # user's original query appended after instruction
    documents=[seg["transcript"] for seg in candidates],
    model="rerank-2.5",
    top_k=10
)
# Reorder candidates by reranking score
reranked = [candidates[r.index] for r in results.results]
```

Demo talking point: "We're not just searching — we're reranking with a cross-encoder that jointly processes the query and each document. And with rerank-2.5, we can steer the reranker with instructions — like telling it to prioritize audience-facing moments over backstage commentary."

Atlas Automated Embedding for content_memory (future optimization)

Automated Embedding in MongoDB Vector Search can auto-generate embeddings using Voyage AI whenever data is inserted or updated. However, **it is currently Public Preview on**

MongoDB Community Edition only — not available on Atlas. Since the demo runs on Atlas, we **cannot use this feature yet.**

What to do instead: Call the Voyage API directly to embed `content_memory` entries on publish. When Automated Embedding ships on Atlas, it will eliminate this code entirely.

```
# On publish, embed the hook_text before writing to content_memory:
embedding = vo.embed(
    texts=[clip["hook_text"]],
    model="voyage-4",
    input_type="document"
).embeddings[0]
clip_doc["embedding"] = embedding
db.content_memory.insert_one(clip_doc)
```

Demo talking point: "Today we call Voyage directly to embed published clips. When Atlas Automated Embedding goes GA, this code disappears — the database handles it natively. That's the direction we're heading: fewer pipelines, more built-in intelligence."

Transcript summaries via Claude

During ingest, after merging frame descriptions with STT transcripts, add a Claude call per segment to produce a 1-2 sentence summary. Stored as `transcript_summary` on each `video_segments` document. This makes cut list output dramatically more readable on a keynote stage.

```
# Per segment during ingest:
summary = claude_client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=100,
    messages=[{
        "role": "user",
        "content": f"Summarize this keynote segment in 1-2 sentences. "
                   f"Speaker: {speaker_label}. "
                   f"Transcript: {transcript}"
    }]
).content[0].text
# Store on the segment document as transcript_summary
```

Before: "And that's why we built Atlas Vector Search to handle exactly this kind of problem where you need to understand the meaning behind what someone is searching for rather than just matching keywords and that's a fundamental shift in how we..."

After: "CJ explains why semantic understanding matters more than keyword matching for modern search."

The summary is also a denser semantic target for search — the Keynote Agent can search against summaries in addition to raw transcripts for better recall.

8. Pre-Seed Data (Complete)

Four projects (shows learning trajectory)

The demo should show a system that's been running across multiple events. Four projects, oldest to newest:

Project	Status	Purpose in demo
A: .local Austin 2025 `proj_austin`	`published`	Earliest event. Generic captions in content_memory show the agent was still learning.
B: .local New York 2025 `proj_nyc`	`published`	Slightly better captions. Agent started picking up patterns.
C: .local SF 2026 `proj_sf`	`published`	Strong captions, well-matched to platforms. Agent has learned the editor's voice. Cross-project memory payoff visible here.
D: .local London 2026 `proj_london`	`in_progress`	Main demo workspace. Agent already has London cuts informed by everything it learned from A, B, C.

Projects A and B only need minimal docs (project_id, title, status, campaign_id). No video files or feedback needed — they exist as history. Project C needs a few recommended_cuts to show in the project list. Project D is the full workspace.

Stakeholder feedback for project D (9 entries)

CJ — CEO (4 comments)

@ 2:34–3:12 "Make sure you get my good side. The left camera angle is better for this segment."

@ 8:15–8:45 "This is the money quote about developer experience. Lead with this for LinkedIn."

@ 14:20–15:00 "Cut this part — I misspoke about the timeline."

@ 22:10–23:30 "The audience reaction was great. Use the wide shot."

Oz — VP Engineering (3 comments)

@ 5:00–6:30 "Resilience section needs to be a standalone clip. Core message for enterprise."

@ 14:20–15:00 **CONFLICT** "Keep this — the correction makes CJ more relatable. Real > polished."

@ 18:00–19:15 "Atlas Vector Search demo — caption must explain what vector search does. No jargon."

Sarah K. — PMM (2 comments)

@ 8:15–8:45 "For Reels, trim to just the last sentence. Setup too long for short-form."

@ 22:10–23:30 "Add text overlay with the key stat. Numbers perform 2x on social."

Pre-generated recommended_cuts for project D (5 clips)

These should be present when the project loads — the agent already ran once based on the 9 feedback entries above.

```
[
  {
```

```
    "clip_id": "cut_001",
    "start_time": 495.0, "end_time": 525.0,
    "transcript_excerpt": "This is what developers actually care about...",
    "speaker": "SPEAKER_01",
    "suggested_caption": {
      "linkedin": "What does it take to build search that understands meaning, not
just keywords?",
      "reels": "This is why vector search changes everything"
    },
    "suggested_hashtags": {
      "linkedin": ["#MongoDB", "#VectorSearch", "#AI", "#DeveloperExperience"],
      "reels": ["#mongodb", "#vectorsearch", "#devtools", "#ai"]
    },
    "asset_recommendation": "Use left camera angle (CJ preference). Tight crop on
speaker with slide visible.",
    "memory_notes": "CJ flagged as money quote. Sarah says trim for Reels. No dupe.
LinkedIn 60s, Reels 15s.",
    "status": "pending"
  },
  {
    "clip_id": "cut_002",
    "start_time": 300.0, "end_time": 390.0,
    "transcript_excerpt": "Resilience isn't optional when your customers depend on
you...",
    "speaker": "SPEAKER_01",
    "suggested_caption": {
      "linkedin": "When your customers depend on you, resilience isn't a feature.
It's the architecture.",
      "reels": "Your database goes down. Your customers don't wait."
    },
    "suggested_hashtags": {
      "linkedin": ["#MongoDB", "#Resilience", "#CloudArchitecture", "#Enterprise"],
      "reels": ["#mongodb", "#database", "#reliability", "#tech"]
    },
    "asset_recommendation": "Wide shot showing demo screen behind speaker. Pair with
architecture diagram overlay.",
    "memory_notes": "Oz requested standalone clip. WARNING: similar clip used in SF
campaign (0.82 similarity). Consider fresh angle.",
    "status": "pending"
  },
  {
```

```
    "clip_id": "cut_003",
    "start_time": 860.0, "end_time": 920.0,
    "transcript_excerpt": "So what does vector search actually do differently...",
    "speaker": "SPEAKER_01",
    "suggested_caption": {
      "linkedin": "Traditional search matches keywords. Vector search understands
what you actually mean.",
      "reels": "Keyword search vs. vector search, explained in 15 seconds"
    },
    "suggested_hashtags": {
      "linkedin": ["#VectorSearch", "#SemanticSearch", "#MongoDB", "#AI"],
      "reels": ["#vectorsearch", "#ai", "#explained", "#learntech"]
    },
    "asset_recommendation": "Screen capture of demo showing search results. Text
overlay with the comparison stat.",
    "memory_notes": "Oz says explain without jargon. Caption follows brand voice. No
dupe.",
    "status": "pending"
  },
  {
    "clip_id": "cut_004",
    "start_time": 1330.0, "end_time": 1410.0,
    "transcript_excerpt": "And the response from the developer community has been
incredible...",
    "speaker": "SPEAKER_01",
    "suggested_caption": {
      "linkedin": "The developer community response to Atlas Vector Search has
exceeded every benchmark we set.",
      "reels": "The numbers are in. Developers are choosing MongoDB for AI."
    },
    "suggested_hashtags": {
      "linkedin": ["#MongoDB", "#DevCommunity", "#AtlasVectorSearch", "#Growth"],
      "reels": ["#mongodb", "#developers", "#ai", "#numbers"]
    },
    "asset_recommendation": "Wide shot with audience visible (CJ prefers audience in
frame). Add text overlay with the key stat Sarah requested.",
    "memory_notes": "CJ noted great audience reaction. Sarah wants stat overlay. No
dupe.",
    "status": "pending"
  },
  {
```

```
    "clip_id": "cut_005",
    "start_time": 154.0, "end_time": 192.0,
    "transcript_excerpt": "Welcome to MongoDB .local London...",
    "speaker": "SPEAKER_01",
    "suggested_caption": {
      "linkedin": "MongoDB .local London 2026 just kicked off. Here's what you
missed.",
      "reels": "We just landed in London. Let's go."
    },
    "suggested_hashtags": {
      "linkedin": ["#MongoDBLocal", "#London", "#DevConference", "#MongoDB"],
      "reels": ["#mongodblocal", "#london", "#tech", "#conference"]
    },
    "asset_recommendation": "Opening energy shot. Wide angle of the venue with crowd
visible. Good for event hype content.",
    "memory_notes": "Opening energy, good for event hype content. No dupe.",
    "status": "pending"
  }
]
```

Content memory across all projects (shows learning trajectory)

Captions get noticeably better from Austin → NYC → SF, showing the agent learning over time.

```
[
  // --- Austin (earliest, generic captions) ---
  {
    "clip_id": "austin_clip_001",
    "project_id": "proj_austin",
    "video_source": "local_austin_2025",
    "hook_text": "Check out highlights from our Austin event!",
    "embedding": [/* 1024 floats */],
    "posted_at": "2025-06-10T10:00:00Z",
    "platform": "linkedin"
  },
  {
    "clip_id": "austin_clip_002",
    "project_id": "proj_austin",
    "video_source": "local_austin_2025",
    "hook_text": "MongoDB .local Austin was a blast. Watch this.",
    "embedding": [/* 1024 floats */],
```

```
    "posted_at": "2025-06-11T14:00:00Z",
    "platform": "reels"
  },

  // --- NYC (better, starting to learn voice) ---
  {
    "clip_id": "nyc_clip_001",
    "project_id": "proj_nyc",
    "video_source": "local_nyc_2025",
    "hook_text": "MongoDB Atlas now handles 10x the throughput. Here's the demo that
proved it.",
    "embedding": [/* 1024 floats */],
    "posted_at": "2025-10-20T10:00:00Z",
    "platform": "linkedin"
  },
  {
    "clip_id": "nyc_clip_002",
    "project_id": "proj_nyc",
    "video_source": "local_nyc_2025",
    "hook_text": "30 seconds on why every AI app needs a vector database",
    "embedding": [/* 1024 floats */],
    "posted_at": "2025-10-21T09:00:00Z",
    "platform": "reels"
  },

  // --- SF (strong captions, agent has learned the voice) ---
  {
    "clip_id": "sf_clip_001",
    "project_id": "proj_sf",
    "video_source": "local_sf_2026",
    "hook_text": "Here's what developers are actually building with agents",
    "embedding": [/* 1024 floats */],
    "posted_at": "2026-03-15T10:00:00Z",
    "platform": "linkedin"
  },
  {
    "clip_id": "sf_clip_002",
    "project_id": "proj_sf",
    "video_source": "local_sf_2026",
```

```
    "hook_text": "Resilience at scale starts with your data layer",
    "embedding": [/* 1024 floats - similar to London cut_002 */],
    "posted_at": "2026-03-16T14:00:00Z",
    "platform": "linkedin"
  },
  {
    "clip_id": "sf_clip_003",
    "project_id": "proj_sf",
    "video_source": "local_sf_2026",
    "hook_text": "15 seconds on why MongoDB chose to acquire Voyage AI",
    "embedding": [/* 1024 floats */],
    "posted_at": "2026-03-17T09:00:00Z",
    "platform": "reels"
  }
]
```

Note: `sf_clip_002` should have high similarity to London `cut_002` so the agent flags the dupe warning. The Austin clips are deliberately generic — this is the "before" in the learning story.

Style memory (accumulated from Austin + NYC + SF)

```
{
  "user_id": "editor_mikiko",
  "preferred_clip_lengths": { "reels": 15, "linkedin": 60, "twitter": 30 },
  "pacing": "fast_cuts",
  "hook_style": "question_first",
  "feedback_patterns": [
    "consistently flags pacing issues for Reels",
    "prefers question-first hooks",
    "favors audience reaction shots over speaker close-ups"
  ],
  "accepted_cuts": ["austin_clip_001", "austin_clip_002", "nyc_clip_001",
"nyc_clip_002", "sf_clip_001", "sf_clip_002", "sf_clip_003"],
  "rejected_cuts": ["austin_draft_003", "nyc_draft_003", "sf_draft_004"],
  "updated_at": "2026-03-17T12:00:00Z"
}
```

9. What to Build

Component 1: Project service

New: `backend/services/project_service.py`

Endpoints:

- `GET /projects` — list all projects (home screen)
- `GET /projects/{project_id}` — load full workspace (project doc + feedback + style_memory + project_memory)
- `POST /projects` — create new project

Component 2: Audio processing + transcript summaries

New: `backend/services/audio_service.py`

Extract audio, transcribe via `gpt-4o-transcribe-diarize`, match to frames by timestamp, store segment boundaries on project doc. **Also generate a 1-2 sentence summary per segment via Claude** — stored as `transcript_summary`. This makes cut list output readable on stage and improves search recall (summaries are denser semantic targets). For large batch ingest, use the Anthropic Batch API at 50% off standard rates.

> **25MB limit + speaker label gotcha:** Split long audio into chunks via ffmpeg. Reassemble timestamps by adding chunk offset. **Critical: speaker labels do NOT persist across chunks.** SPEAKER_A in chunk 1 may be a different person than SPEAKER_A in chunk 2. Fix by supplying `known_speaker_names` + `known_speaker_references` (2-10s reference audio clips per speaker, up to 4 speakers) on each chunk request, or implement a post-alignment step that matches speakers across chunks by voice similarity. For pre-baked demo data, manually verify the labels.

Component 3: Search (Keynote Agent) with reranking

Endpoint: `POST /projects/{project_id}/search`

Wraps existing search logic as a LangGraph tool. Pipeline: embed query via Voyage → route to hybrid/vector/text search → **rerank top candidates with Voyage rerank-2.5** → return results. The reranking step is a cross-encoder that jointly processes the query and each candidate for more accurate relevance than embedding similarity alone. Returns matching segments as a thumbnail strip. Clicking a result jumps the video and optionally adds the segment to the candidate clip list.

Component 4: Feedback service

New: `backend/services/feedback_service.py`

- `POST /projects/{project_id}/feedback` — store + auto-trigger Social Agent re-generation
- `GET /projects/{project_id}/feedback` — all feedback sorted by timestamp
- `GET /projects/{project_id}/feedback/at/{timestamp}` — feedback covering a specific timestamp

Timestamp range default: When the editor submits feedback, the timestamp range defaults to the segment boundary covering the current playhead position (from `project.segment_boundaries`). Frontend sends the current playhead time; backend looks up the covering segment. Editor can optionally drag start/end handles on the timeline to override.

Component 5: Style reference service

New: `backend/services/style_ref_service.py`

`POST /projects/{project_id}/style-reference`

Upload clip → extract frames (1/s) → embed via Voyage → Claude analyzes style → store on project doc → trigger re-generation.

Style matching is limited to: pacing/timing recs, caption tone, format suggestions. Not visual transformation.

> **Voyage multimodal limits:** Max 20MB per video, max 16M pixels per image, max 32k tokens per input. Most Instagram/TikTok clips are well under 20MB. If a reference clip exceeds the limit, ffmpeg-compress it before embedding.

Component 6: Agent service (LangGraph)

New: `backend/services/agent_service.py`

Social Agent triggers: (1) feedback submitted, (2) style reference uploaded, (3) `POST /projects/{project_id}/regenerate` manual button.

Uses system prompt from section 6. On each run, also calls the feedback pattern detection prompt and writes to `style_memory.feedback_patterns`.

Component 7: Caption editing

`PATCH /projects/{project_id}/cuts/{clip_id}`

Saves edited captions back to `project.recommended_cuts[n].suggested_caption`. Simple field update.

Component 8: Publish service

New: `backend/services/publish_service.py`

`POST /projects/{project_id}/publish` — body: list of approved clip IDs.

Actions: write to `published_clips`, add to `content_memory` (embed `hook_text` via Voyage `voyage-4` API call, then insert), update `style_memory`, set project status, return confirmation.

10. UI Specification

The workspace has a 4-zone layout: search bar + video/timeline + feedback panel + cut list on the left, memory sidebar on the right.

Figure 11: Project workspace wireframe — video player with feedback markers, search results, cut list with approve/reject, memory sidebar

Behavior	Implementation
Search results as thumbnail strip	Above video player. Clicking a result seeks the video to that timestamp.
Feedback syncs with scrubbing	onTimeUpdate filters feedback by timestamp range. Markers highlight when playhead is within range.
Timestamp range defaults to segment	Frontend sends playhead time. Backend maps to segment_boundaries.
Re-generation loading state	Old cut list dims (opacity 0.5) with spinner overlay. WebSocket pushes progress. Target: <5s.
Captions are editable	Inline text inputs on each clip card. Blur/enter saves via PATCH endpoint.
Publish shows confirmation	Toast: '3 clips sent to social team. Content memory updated.'
Memory sidebar reflects cross-project state	Shows 'loaded from SF' next to prefs. Updates live after publish.

11. Latency & Fallback (10 optimizations)

Operation	Naive	Optimized	How
Project load	2-4s	< 1s	Warm connection pool on startup
Search	3-4s	< 2s	Skip Claude router, hardcode hybrid
Re-generation	8-12s	< 5s	Merge prompts, stream, cache, code-only conflicts
Style reference	12-19s	< 6s	Pre-process reference clip, only re-gen is live
Publish	1-2s	< 1s	Already fast, parallel writes

Key optimizations: (1) Merge three Claude calls into one merged prompt with conflicts + patterns + cuts in a single JSON response. (2) Skip Claude for search routing — hardcode hybrid search. (3) Code-only conflict detection with keyword heuristics. (4) Stream Claude's response via SSE so first clip appears in ~3s. (5) Anthropic prompt caching on system prompt + memory context. (6) Pre-warm Social Agent on project load. (7) Pre-process the demo style reference clip. (8) Connection pooling + parallel I/O with asyncio.gather. (9) Use Haiku for fast-path calls. (10) Set max_tokens tightly.

Implement DEMO_FALLBACK_MODE env var. When true, all external API calls return cached data. The presenter can flip this backstage if network is unreliable. Also implement per-call timeouts with fallback to cached results.

12. MongoDB Indexes

Collection	Index	Type	Status
`video_segments`	`vector_search_index`	Vector 1024d cosine on `embedding`	`exists`
`video_segments`	`text_search_index`	Full-text: `description, transcript, transcript_summary`	`modify`
`content_memory`	`content_vector_idx`	Vector 1024d dotProduct on `embedding`	`new`
`stakeholder_feedback`	`feedback_by_project`	Compound: `project_id` + `timestamp_range.start`	`new`
`projects`	`project_by_status`	Index on `status`	`new`

13. Stack & Dependencies

Component	Tool	Status
Database	MongoDB Atlas ($vectorSearch GA)	exists
Video embeddings	voyage-multimodal-3.5 (1024d, 32k tok, 20MB)	exists
Text embeddings	voyage-4 (1024d, shared space, $0.06/MTok)	new
Reranking	rerank-2.5 (32k ctx, instruction-following)	new
STT + Diarization	gpt-4o-transcribe-diarize (GA, 25MB limit)	new
LLM	claude-sonnet-4-6 (GA, 1M ctx, $3/$15)	new
Agent framework	LangGraph 1.1.x	new
Checkpointer	langgraph-checkpoint-mongodb 0.3.1	new
Long-term memory	langgraph-store-mongodb 0.2.0	new

Pip dependencies (pin exactly)

```
anthropic>=0.90,<1.0
voyageai==0.3.7
langgraph==1.1.5
langgraph-prebuilt==1.0.9        # pin explicitly
langgraph-checkpoint-mongodb==0.3.1
langgraph-store-mongodb==0.2.0
langchain-core>=0.3,<1.0
```

Embedding space compatibility: voyage-multimodal-3.5 and Voyage 4 are in different embedding spaces. Cannot mix in the same index. video_segments uses multimodal-3.5, content_memory uses voyage-4. Within Voyage 4, models ARE interchangeable.

14. Files to Modify & Create

Modify

File	Changes
`backend/main.py`	Add all new endpoints. **Remove `video_path.unlink()` so video files persist for playback.**
`backend/services/ai_service.py`	Add transcribe_audio, Claude client, style analysis, feedback pattern detection
`backend/services/mongodb_service.py`	Project CRUD, feedback CRUD, memory CRUD, content vector search, published_clips
`backend/services/video_processor.py`	Audio extraction, transcript + speaker_label on frames, segment_boundaries output
`backend/models/schemas.py`	Add: Project, FeedbackEntry, StyleReference, SuggestedClip, CutListResponse, PublishRequest, SearchResult
`backend/requirements.txt`	anthropic, langgraph, langchain-core, langgraph-checkpoint-mongodb, langgraph-store-mongodb

Create

File	Purpose
`backend/services/project_service.py`	Project CRUD, workspace loader
`backend/services/audio_service.py`	Audio extraction + STT + diarization
`backend/services/feedback_service.py`	Feedback CRUD, conflict detection, segment boundary lookup, re-generation trigger
`backend/services/style_ref_service.py`	Style reference upload + embed + Claude analysis
`backend/services/agent_service.py`	LangGraph agents, MongoDBSaver, HITL middleware, pattern detection
`backend/services/publish_service.py`	Publish flow + memory updates
`frontend/src/pages/ProjectList.jsx`	Home screen project cards
`frontend/src/pages/ProjectWorkspace.jsx`	Main workspace layout
`frontend/src/components/SearchBar.jsx`	Query input + thumbnail strip results
`frontend/src/components/VideoTimeline.jsx`	Video player with feedback markers + segment boundary scrubbing
`frontend/src/components/FeedbackPanel.jsx`	Timestamp-synced feedback + conflict flags
`frontend/src/components/FeedbackInput.jsx`	Comment input with segment-defaulted timestamp range + draggable handles
`frontend/src/components/StyleReferenceUpload.jsx`	Drag-drop for reference clips
`frontend/src/components/CutListPanel.jsx`	Clip cards with editable captions, approve/reject, memory notes
`frontend/src/components/PublishButton.jsx`	Publish + confirmation toast
`frontend/src/components/MemorySidebar.jsx`	Memory status, learned prefs, trace, brand voice

15. What NOT to Build

- No video rendering/cutting — output is a cut list, not mp4 files
- No visual style transformation — style matching is timing + copy tone only
- No guardrails/content safety — deferred to v2
- No TTS voiceover — deferred
- No real-time streaming audio — pre-baked during ingest
- No external publish integrations — write to published_clips + show confirmation
- No multi-user auth — user_id is hardcoded as 'editor_mikiko'
- No agent auto-resolution of conflicts — editor always decides manually

Confidential — MongoDB Internal